

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
SUMMER 2024**



**FIRECRACKERS
FIRECRACKER**

**SAMIUL ISLAM
JOSUE GOMEZ
ASMIN POTHULA
HUNG VAN LE
KHANH HA**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	08.01.2024	SI	document creation
0.2	08.05.2024	AP, KH	complete draft
0.3	08.05.2024	AP, JG	release candidate 1
1.0	08.06.2024	SI, JG, HL	official release
1.1	08.06.2024	HL	added design review requests

CONTENTS

1	Introduction	5
1.1	Concept	5
1.2	Scope	5
1.3	Key Requirements	5
2	System Overview	6
2.1	Presentation Layer Description	6
2.2	Business Logic Layer Description	6
2.3	Data Layer Description	6
2.4	Security Layer Description	7
2.5	Communication Layer Description	7
3	Subsystem Definitions & Data Flow	8
4	Presentation Layer Subsystems	9
4.1	Subsystem 1: UI Components Subsystem	9
4.2	Subsystem 2: User Interaction Handling Subsystem	9
5	Business Logic Layer Subsystems	11
5.1	Subsystem 1: Task Manager Subsystem	11
5.2	Subsystem 2: Sprint Manager Subsystem	12
5.3	Subsystem 3: Report Generator Subsystem	12
6	Data Layer Subsystems	14
6.1	Subsystem 1: Database Management Subsystem	14
6.2	Subsystem 2: Data Retrieval and Storage Subsystem	14
7	Security Layer Subsystems	16
7.1	Authentication Subsystem	16
7.2	Authorization Subsystem	17
7.3	Data Encryption Subsystem	18
8	Communication Layer Subsystems	19
8.1	API Management Subsystem	19
8.2	Request Handling Subsystem	20
8.3	Subsystem 3	21

LIST OF FIGURES

1	Firecracker architectural layer diagram	6
2	A simple data flow diagram	8
3	Presentation subsystem description diagram	9
4	Presentation subsystem description diagram	10
5	Presentation subsystem description diagram	11
6	Presentation subsystem description diagram	12
7	Presentation subsystem description diagram	13
8	Presentation subsystem description diagram	14
9	Presentation subsystem description diagram	15
10	Presentation subsystem description diagram	16
11	Presentation subsystem description diagram	17
12	Presentation subsystem description diagram	18
13	Example subsystem description diagram	19
14	Example subsystem description diagram	20
15	Example subsystem description diagram	21

LIST OF TABLES

2	Subsystem interfaces	9
3	Subsystem interfaces	10
4	Subsystem interfaces	11
5	Subsystem interfaces	12
6	Subsystem interfaces	13
7	Subsystem interfaces	14
8	Subsystem interfaces	15
9	Subsystem interfaces	16
10	Subsystem interfaces	17
11	Subsystem interfaces	18
12	Subsystem interfaces	19
13	Subsystem interfaces	20
14	Subsystem interfaces	21

1 INTRODUCTION

1.1 CONCEPT

The project management web application, named Firecracker, is designed to streamline and enhance the management of senior design projects in a college setting. The application aims to provide an integrated platform that supports project and sprint backlog management, task tracking, progress monitoring, and collaboration among team members, professors, and sponsors. Firecracker will serve as a comprehensive tool that aids in the organization and execution of senior design projects by offering features such as sprint planning, task estimation, daily updates, burndown chart generation, and sprint retrospectives.

1.2 SCOPE

The scope of Firecracker encompasses the following key functionalities:

- **Project and Sprint Management:** Creation and management of teams, projects, and sprint schedules, including setting start/end dates and presentation dates.
- **Backlog Management:** Addition, modification, and prioritization of product and sprint backlog items.
- **Task Management:** Breakdown of backlog items into tasks, estimation of time requirements, task assignments, and progress tracking.
- **Burndown Chart Generation:** Visualization of sprint progress through burndown charts.
- **Sprint Reporting and Retrospectives:** Generation of detailed sprint reports, including peer reviews, and facilitation of sprint retrospectives to assess and improve processes.
- **Multiple Views:** Provision of different views such as Scrum board, calendar, table, and instructor-specific views for varied user needs.

1.3 KEY REQUIREMENTS

The Firecracker web application will be designed with the following key requirements:

- **User Interface:** A user-friendly interface for seamless interaction and navigation.
- **Data Security:** Secure handling of user data and project information through authentication, authorization, and encryption.
- **Scalability:** Design to handle varying project sizes and numbers of users efficiently.
- **Performance:** Efficient performance with minimal latency in task updates and data retrieval.

2 SYSTEM OVERVIEW

This section should describe the overall structure of your software system. Think of it as the strategy for how you will build the system. An architectural "layer" is the top-level logical view, or an abstraction, of your design. Layers should be composed of related elements of similar capabilities, and should be highly independent of other layers, but should have very clearly defined interfaces and interactions with other layers. Each layer should be identified individually and should be unique as to its function and purpose within the system. This section should also contain the high-level block diagram of the layers, as shown in the example below, as well as detailed descriptions of the functions of each layer.

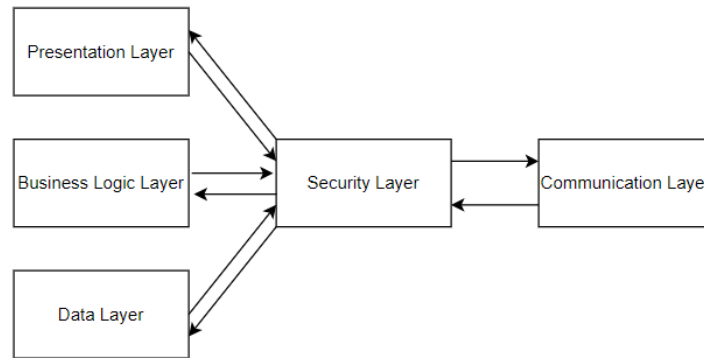


Figure 1: Firecracker architectural layer diagram

2.1 PRESENTATION LAYER DESCRIPTION

The Presentation Layer is responsible for the user interface and user experience. It is the front-end of the application where users interact with the system. This layer includes various UI components that provide visual elements and interactive features, enabling users to perform tasks efficiently. It handles user interactions by managing user inputs and events, sending them to the Business Logic Layer for processing. Additionally, it receives data from the Business Logic Layer and displays it to the user in a comprehensible format, ensuring an intuitive and responsive user experience.

2.2 BUSINESS LOGIC LAYER DESCRIPTION

The Business Logic Layer is the core functionality hub of the application, encompassing the business rules and logic required to fulfill user requests and perform operations. This layer is responsible for task management, including creating, updating, and deleting project elements. It also handles sprint management, which involves the creation and tracking of sprints, task assignments, and progress monitoring. Moreover, the Business Logic Layer compiles and generates reports on project and sprint progress, providing valuable insights for users. It processes user requests and communicates with the Presentation and Data Layers to ensure accurate and efficient operation.

2.3 DATA LAYER DESCRIPTION

The Data Layer is tasked with data storage, retrieval, and management, ensuring data integrity and providing data services to other layers. It manages the database where project data is stored, handling all read and write operations. This layer is also responsible for data validation, ensuring data consistency and integrity before storage. By managing data efficiently and securely, the Data Layer supports the overall functionality of the application and enables seamless data access and management for the Business Logic and Presentation Layers.

2.4 SECURITY LAYER DESCRIPTION

The Security Layer is dedicated to protecting the system from unauthorized access and vulnerabilities. It manages authentication and authorization processes, verifying the identity of users before granting access and managing user permissions and access levels. Additionally, the Security Layer secures data through encryption, both at rest and in transit. By implementing robust security measures, this layer ensures that the application remains secure and that sensitive information is protected from potential threats.

2.5 COMMUNICATION LAYER DESCRIPTION

The Communication Layer handles the interaction between the client-side and server-side of the application, managing network communication and API requests. It includes API management, which involves overseeing API endpoints for client-server communication. This layer processes incoming requests, routing them to the appropriate services, and handles the responses, sending them back to the client after processing. By efficiently managing the flow of data between the client and server, the Communication Layer ensures that the application operates smoothly and responsively.

3 SUBSYSTEM DEFINITIONS & DATA FLOW

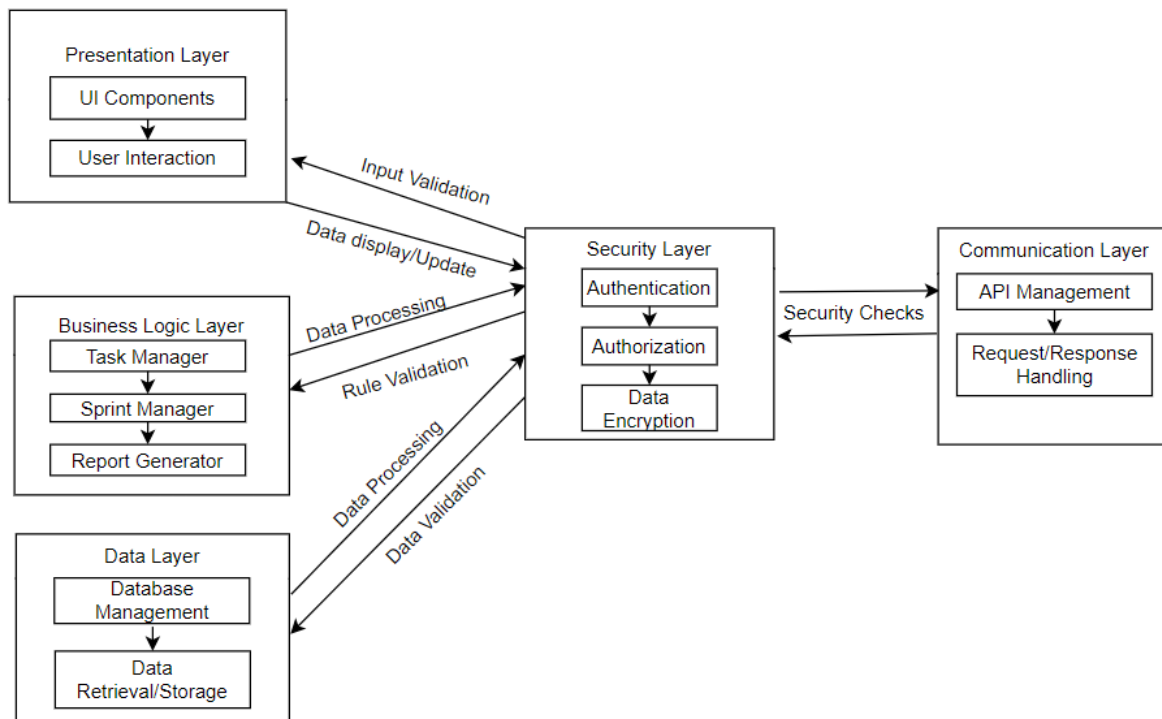


Figure 2: A simple data flow diagram

4 PRESENTATION LAYER SUBSYSTEMS

4.1 SUBSYSTEM 1: UI COMPONENTS SUBSYSTEM

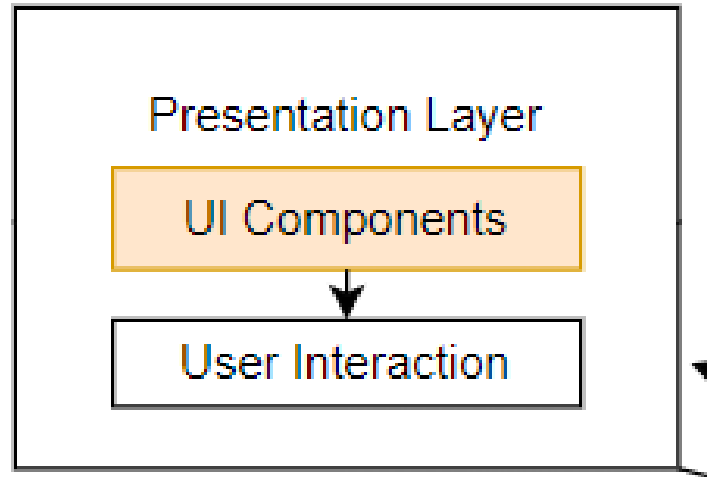


Figure 3: Presentation subsystem description diagram

4.1.1 ASSUMPTIONS

- The UI Components Subsystem assumes that users will interact with the application via a web browser.

4.1.2 RESPONSIBILITIES

- The subsystem is responsible for rendering all visual elements such as buttons, forms, modals, and other UI elements.

4.1.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 2: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	User Input Interface: Captures user actions (clicks, typing, etc.) and translates them into events.	User actions	User events
#02	Data Display Interface	Processed data	UI updates

4.2 SUBSYSTEM 2: USER INTERACTION HANDLING SUBSYSTEM

4.2.1 ASSUMPTIONS

- It assumes each user has a unique identifier and will authenticate using secure methods like passwords or biometrics

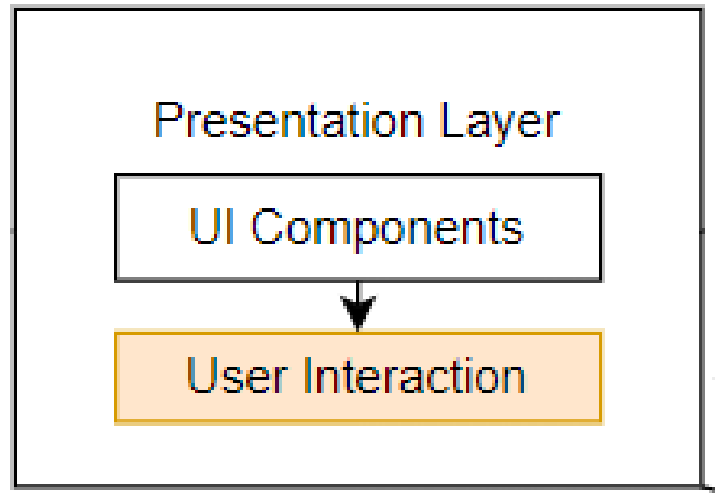


Figure 4: Presentation subsystem description diagram

4.2.2 RESPONSIBILITIES

- Manages user inputs and events, sending them to the Business Logic Layer for processing.
- Ensures smooth interaction between the user and the application.
- Captures and manages user inputs and events, sending them to the Business Logic Layer for processing.

4.2.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 3: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	User Interaction: Interprets and processes user inputs, managing the flow and logic of user interactions across different UI components.	User actions	User events
#02	User Feedback: Provides real-time feedback to users based on their interactions, ensuring a responsive and dynamic user experience.	User interactions and system state changes.	Visual, auditory, or tactile feedback signals.

5 BUSINESS LOGIC LAYER SUBSYSTEMS

5.1 SUBSYSTEM 1: TASK MANAGER SUBSYSTEM

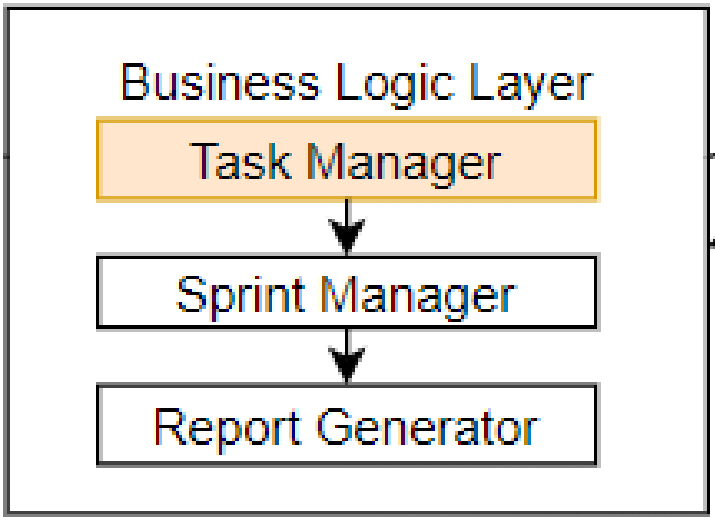


Figure 5: Presentation subsystem description diagram

5.1.1 ASSUMPTIONS

- The Task Manager Subsystem assumes that it will receive tasks as structured data (e.g., JSON objects) and will handle these tasks based on pre-defined business rules and user roles.

5.1.2 RESPONSIBILITIES

- This subsystem is responsible for tracking, updating, and managing the lifecycle of tasks within the application. It handles task assignments, progress updates, and completion statuses.

5.1.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 4: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Task Input Interface: Receives tasks and task-related commands.	Task creation and modification commands.	Status updates.
#02	Task Reporting Interface: Provides comprehensive reporting on task status.	Queries for task details	Task reports

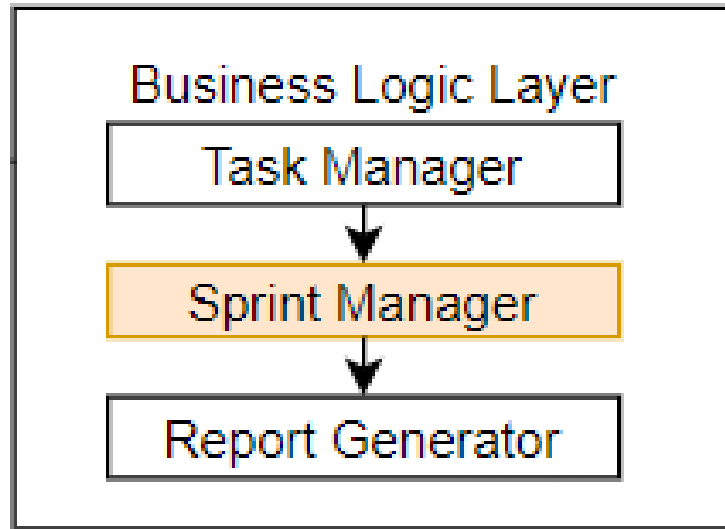


Figure 6: Presentation subsystem description diagram

5.2 SUBSYSTEM 2: SPRINT MANAGER SUBSYSTEM

5.2.1 ASSUMPTIONS

- It assumes that it operates within an agile project management framework, receiving sprint-related data and user interactions for managing sprint activities.

5.2.2 RESPONSIBILITIES

- This subsystem is responsible for organizing, initiating, and tracking the progress of sprints. It manages sprint planning, active sprint monitoring, and sprint review processes.

5.2.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 5: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Sprint Planning Interface: Facilitates the planning phase of sprints	Task lists, sprint goals.	Planned sprint details
#02	Sprint Monitoring Interface: Tracks ongoing sprint progress.	Progress updates, task completion reports.	Updated sprint status
#03	Sprint Review Interface: Supports post-sprint activities.	Completed tasks, sprint feedback.	Sprint review reports

5.3 SUBSYSTEM 3: REPORT GENERATOR SUBSYSTEM

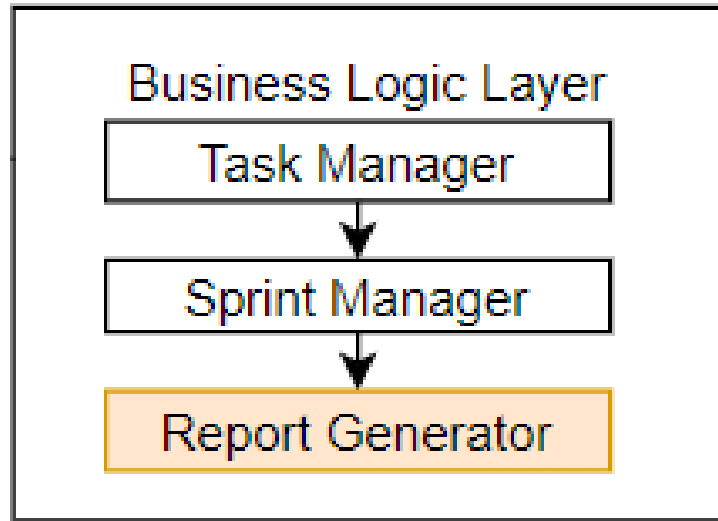


Figure 7: Presentation subsystem description diagram

5.3.1 ASSUMPTIONS

- It assumes it will receive data from various subsystems and user inputs, and it is expected to generate reports in multiple formats such as PDF, Excel, and web-based dashboards.

5.3.2 RESPONSIBILITIES

- This subsystem is responsible for aggregating data, analyzing trends, and generating clear, concise, and informative reports for different students.

5.3.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 6: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Data Collection Interface: Receives data inputs.	Data queries, data requests.	Data sets
#02	Report Distribution Interface: Manages the distribution of generated reports.	Distribution lists, security credentials.	Notifications of report availability.

6 DATA LAYER SUBSYSTEMS

6.1 SUBSYSTEM 1: DATABASE MANAGEMENT SUBSYSTEM

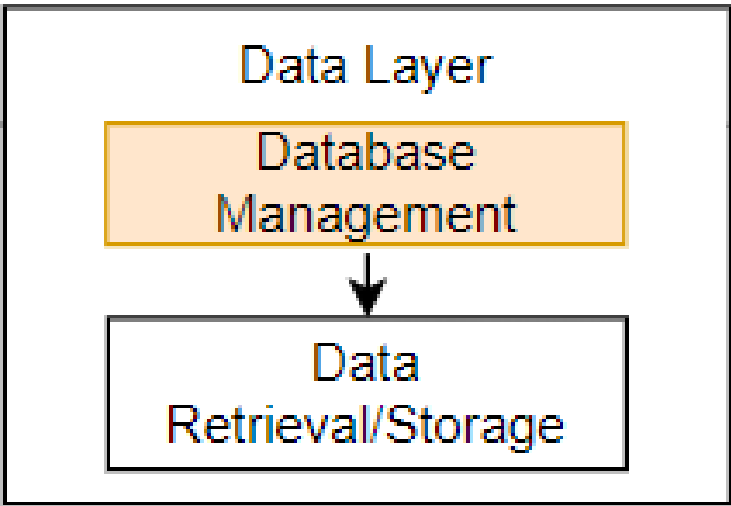


Figure 8: Presentation subsystem description diagram

6.1.1 ASSUMPTIONS

- The Database Management Subsystem assumes it will handle a high volume of transactions and queries, maintaining data integrity, consistency, and security across all operations.

6.1.2 RESPONSIBILITIES

- Ensures efficient data handling, supports transaction processing, and enforces data security measures.

6.1.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 7: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Data Maintenance Interface: Handles routine data maintenance tasks.	Maintenance commands, scheduling parameters.	Maintenance status reports

6.2 SUBSYSTEM 2: DATA RETRIEVAL AND STORAGE SUBSYSTEM

6.2.1 ASSUMPTIONS

- The Data Retrieval and Storage Subsystem assumes reliable connectivity and integration with the Database Management Subsystem for efficient data handling.

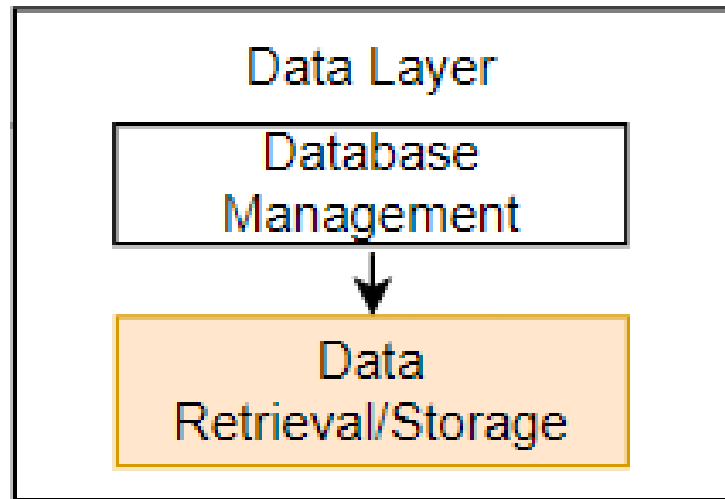


Figure 9: Presentation subsystem description diagram

6.2.2 RESPONSIBILITIES

- This subsystem is responsible for the efficient storage of data, ensuring its availability and integrity, and for the quick retrieval of data to meet user and system requests, supporting real-time data needs and analytical processes.

6.2.3 SUBSYSTEM INTERFACES

Each interface is designed to facilitate communication between the UI Components Subsystem and other subsystems within the architecture.

Table 8: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Data Storage Interface: Manages the storage of data	Data records	Confirmation of data storage
#02	Data Retrieval Interface: Provides mechanisms for querying and retrieving data	Query parameters, access credentials.	Requested data sets, query execution status.

7 SECURITY LAYER SUBSYSTEMS

7.1 AUTHENTICATION SUBSYSTEM

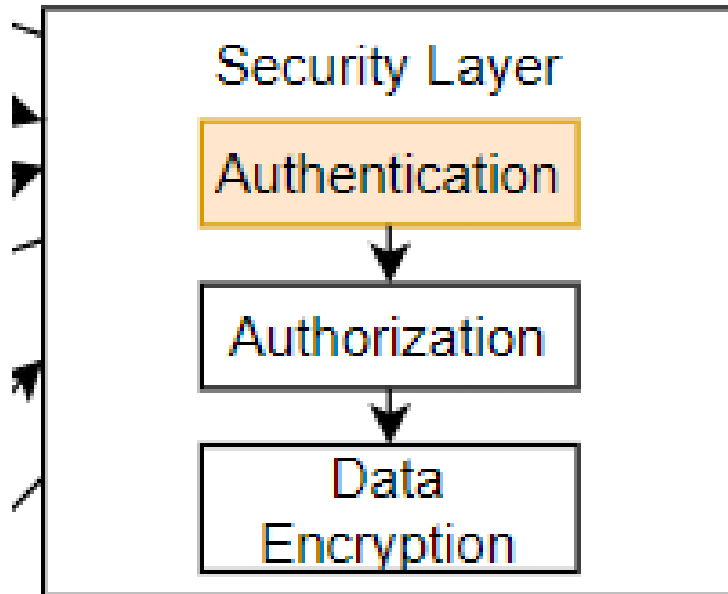


Figure 10: Presentation subsystem description diagram

7.1.1 ASSUMPTIONS

- Users will have unique identifiers for authentication purposes.
- Secure mechanisms are in place for storing and transmitting authentication credentials.
- Interaction with the user interface for login and logout processes is required.

7.1.2 RESPONSIBILITIES

- **User Identity Verification:** Ensure users are authenticated through secure login processes.
- **Credential Management:** Securely store and manage user credentials.
- **Session Management:** Handle user sessions to maintain authentication state.

7.1.3 SUBSYSTEM INTERFACES

Table 9: Subsystem interfaces

ID	Description	Inputs	Outputs
#09	Authentication Interface	Login Requests	Auth Tokens
#15	Credential Storage Interface	User Credential	Storage Confirmation
#16	Session Management Interface	Auth Tokens	Session Data

7.2 AUTHORIZATION SUBSYSTEM

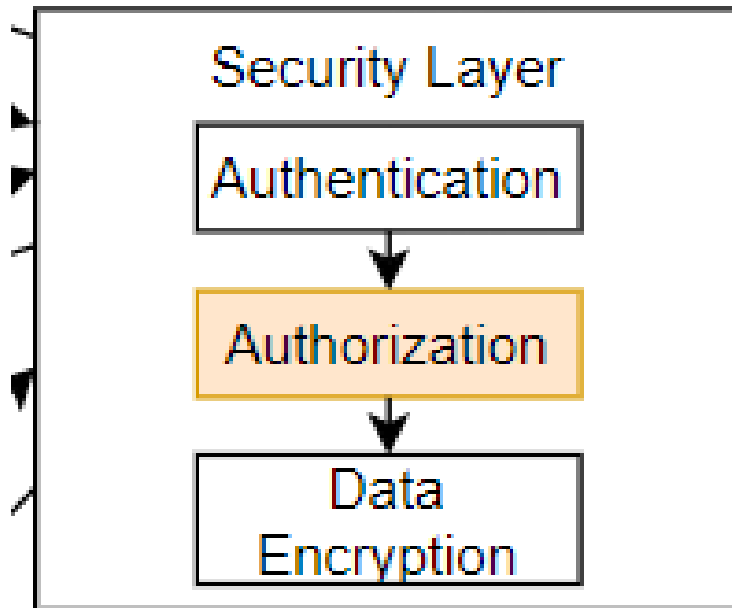


Figure 11: Presentation subsystem description diagram

7.2.1 ASSUMPTIONS

- User roles and permissions are predefined and stored securely.
- Access control mechanisms are implemented based on roles and permissions.

7.2.2 RESPONSIBILITIES

- **Permission Management:** Define and manage permissions for different user roles.
- **Access Control:** Ensure users access only the resources they are authorized for.
- **Role Assignment:** Assign roles to users based on their responsibilities.

7.2.3 SUBSYSTEM INTERFACES

Table 10: Subsystem interfaces

ID	Description	Inputs	Outputs
#10	Authorization Interface	Access Requests	Permission Grants
#17	Role Management Interface	Role Data	Role Assignments
#18	Permission Check Interface	Resource Requests	Access Decisions

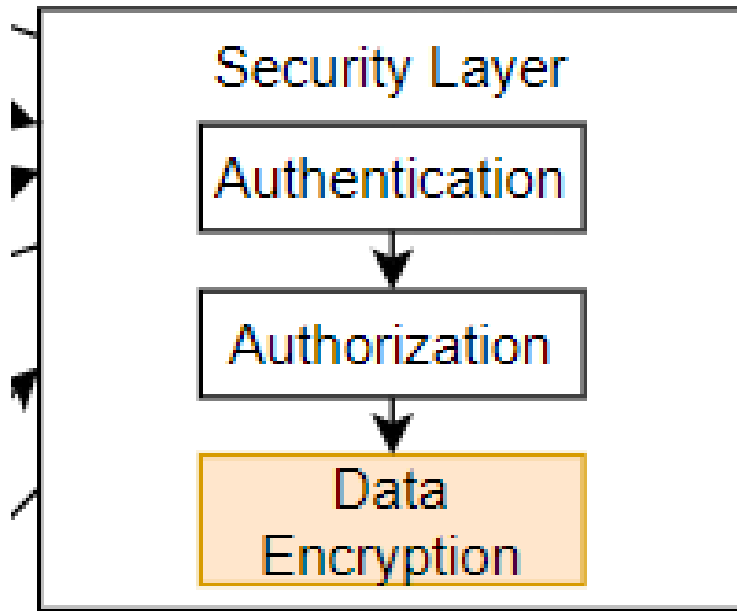


Figure 12: Presentation subsystem description diagram

7.3 DATA ENCRYPTION SUBSYSTEM

7.3.1 ASSUMPTIONS

- Data at rest and in transit requires encryption.
- Encryption keys are securely managed and stored.

7.3.2 RESPONSIBILITIES

- **Data Encryption:** Encrypt data before storage and during transmission.
- **Key Management:** Handle the generation, storage, and rotation of encryption keys.
- **Decryption:** Decrypt data for authorized users and processes.

7.3.3 SUBSYSTEM INTERFACES

Table 11: Subsystem interfaces

ID	Description	Inputs	Outputs
#11	Data Encryption Interface	Plain Data	Encrypted Data
#19	Key Management Interface	Key Requests	Encryption Keys
#20	Data Decryption Interface	Encrypted Data	Plain Data

8 COMMUNICATION LAYER SUBSYSTEMS

8.1 API MANAGEMENT SUBSYSTEM

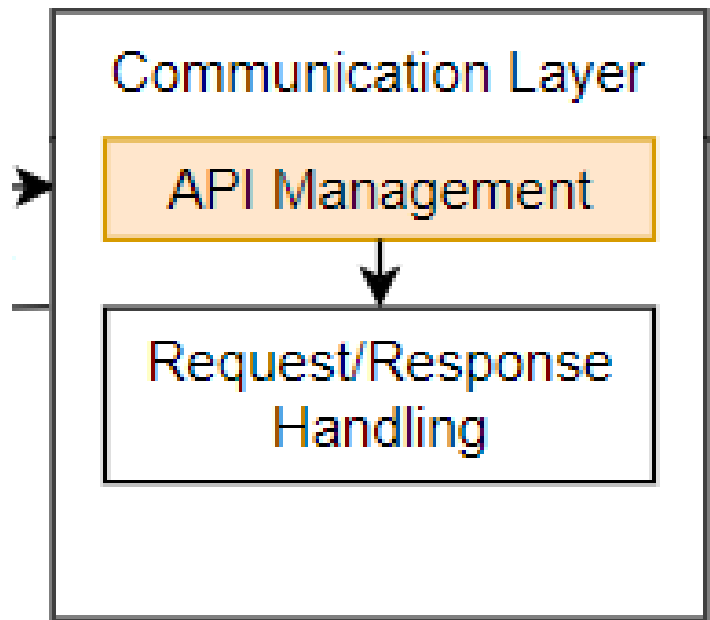


Figure 13: Example subsystem description diagram

8.1.1 ASSUMPTIONS

- APIs are the primary mode of communication between the client and server.
- APIs need to be well-documented and version-controlled.

8.1.2 RESPONSIBILITIES

- **Endpoint Management:** Define and manage API endpoints.
- **Version Control:** Handle different versions of APIs to ensure backward compatibility.
- **Documentation:** Provide clear and comprehensive API documentation for developers.

8.1.3 SUBSYSTEM INTERFACES

Table 12: Subsystem interfaces

ID	Description	Inputs	Outputs
#12	API Request Interface	API Calls	Processed Re-requests
#21	Endpoint Management Interface	Endpoint Data	API Endpoints
#22	Documentation Interface	API Specs	Documentation

8.2 REQUEST HANDLING SUBSYSTEM

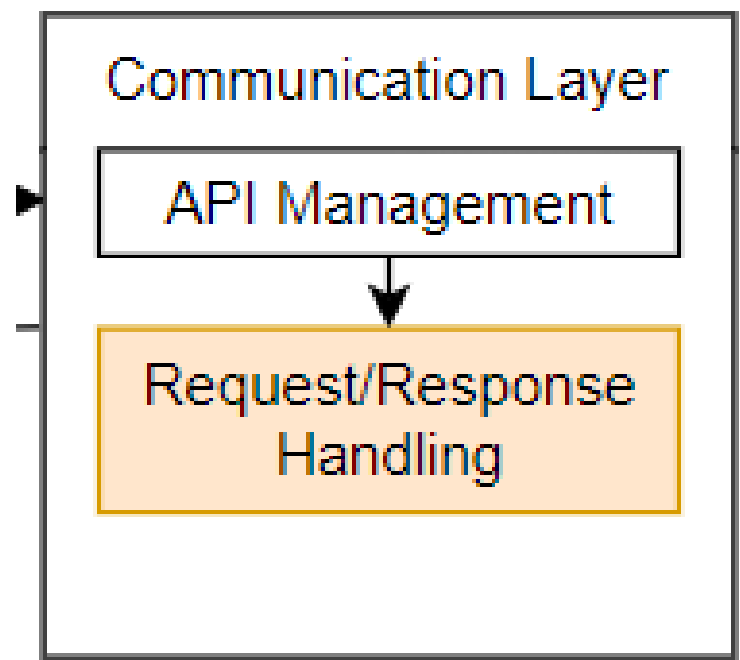


Figure 14: Example subsystem description diagram

8.2.1 ASSUMPTIONS

- Requests need to be efficiently processed and routed to appropriate services.
- The system assumes a reliable network for request transmission.

8.2.2 RESPONSIBILITIES

- **Request Processing:** Handle incoming API requests and route them to the correct services.
- **Error Handling:** Manage errors and exceptions during request processing.
- **Load Balancing:** Distribute requests evenly across server resources.

8.2.3 SUBSYSTEM INTERFACES

Table 13: Subsystem interfaces

ID	Description	Inputs	Outputs
#13	Request Routing Interface	Incoming Requests	Routed Requests
#23	Error Handling Interface	Error Logs	Error Responses
#24	Load Balancing Interface	Requests	Balanced Load

Figure 15: Example subsystem description diagram

8.3 SUBSYSTEM 3

8.3.1 ASSUMPTIONS

- Responses need to be timely and accurate.
- The system assumes a standard format for responses.

8.3.2 RESPONSIBILITIES

- **Response Generation:** Generate appropriate responses after processing requests.
- **Data Formatting:** Ensure responses are in a consistent and readable format.
- **Latency Management:** Minimize latency in delivering responses to clients.

8.3.3 SUBSYSTEM INTERFACES

Table 14: Subsystem interfaces

ID	Description	Inputs	Outputs
#14	Response Handling Interface	Service Responses	Client Responses
#25	Data Formatting Interface	Raw Data	Formatted Re- sponses
#26	Latency Management Interface	Processed Data	Timely Responses

REFERENCES